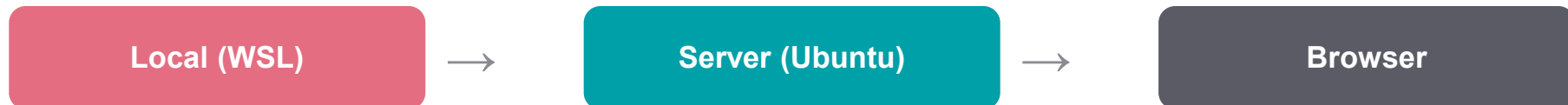


Deploying to SAKURA Cloud

Publishing a Flask-based RAG Web application on a SAKURA Cloud server (Ubuntu)

Reference material for implementing this in the upcoming PBL course



Purpose and Assumptions of This Material

■ Purpose

Deploy the locally running Flask-based RAG Web application to a SAKURA Cloud Ubuntu server, making it accessible from outside.

■ Goal

A state where the application on the server returns HTTP responses, showing an answer and its sources for each question.

■ Audience

Learners with basic Python knowledge. Linux/server work centers on running commands.

■ Assumptions

- Already verified working locally (WSL/Ubuntu) via python app.py on 127.0.0.1:5000
- Structure: app.py / templates/ / static/ / requirements.txt
- The account token is supplied via the AI_ENGINE_TOKEN environment variable and never written in code
- Requirements needed for production use are out of scope here
 - — organized at the end

Local vs. Server Environment

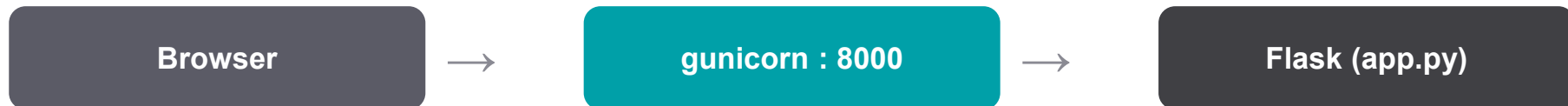
Local (Development)

- **How it runs:** Flask dev server (python app.py)
- **Listens on:** 127.0.0.1 (localhost only)
- **Used by:** The developer themselves

Server (Public)

- **How it runs:** WSGI server (gunicorn)
- **Listens on:** 0.0.0.0 (all interfaces = reachable externally)
- **Used by:** External users
- **Process:** Must persist (out of scope for this material)

The communication path this material builds



→ requests → SAKURA AI Engine (documents_chat) → response

Distinguishing Which Machine You're On

This guide operates on two machines: your local PC and a server on SAKURA Cloud. Always stay aware of which one each command runs on. Tell them apart from the shell prompt.

Prompt Example	Target	Description
<username>@<PC name>:~\$	Local (WSL)	Ubuntu (WSL) on your own PC
ubuntu@<server's global IP>:~\$	Server (Ubuntu)	Ubuntu on SAKURA Cloud, connected via SSH

This material labels each step's execution environment in the top right.

Run: Local (WSL)

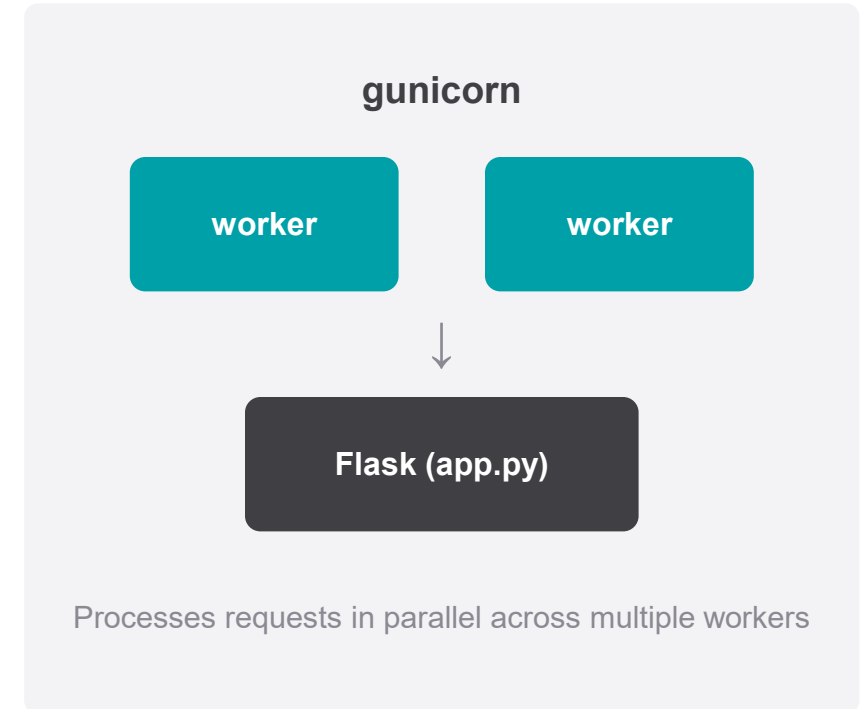
Run: Server (Ubuntu)

Check: Browser

Action: Control Panel

WSGI Servers and gunicorn

- **WSGI:** Web Server Gateway Interface (PEP 3333). The standard interface connecting Python Web applications to a Web server.
- **Flask's built-in dev server:** Meant for debugging; not recommended for production.
- **gunicorn:** A WSGI-compliant production server. Processes requests in parallel across multiple worker processes.
- **app:app:** Notation specifying the application instance app inside the module app (app.py).
- The application code itself doesn't change — only how it's started.



Step 1: Create the Ubuntu Server

Action: Control Panel

Log in to the Cloud Control Panel (secure.SAKURA.ad.jp/cloud/), select your project and zone, then go to [Server] → [Add]. Configure the following in Simple mode.

- **Disk image:** Ubuntu Server 24.04 LTS (verified working with this)
- **Server plan / disk:** Choose based on your testing needs (e.g. 1 core, 1-2GB, 20GB SSD)
- **Network to connect to:** Internet
- **Public key: Don't register one** (The admin user is created as ubuntu)
- **Hostname:** Set any name you like, then run [Create]

Once creation finishes, check the global IP address on the server details screen.

Step 2: SSH Into the Server

Run: Local (WSL)

```
ssh ubuntu@<server's global IP>
```

- Connects to the server using the registered public key. On first connection, type yes to confirm the host key fingerprint.
- Once connected, the prompt changes to ubuntu@... — from here on, you're operating on the server.

Note: To specify a private key explicitly, use `ssh -i ~/.ssh/<private key> ubuntu@<IP>`.

Step 3: Set Up the Python Runtime

Run: Server (Ubuntu)

```
sudo apt update  
sudo apt install -y python3-venv python3-pip
```

- Updates the package index, then installs the virtual environment module (venv) and pip.
- Python itself is already included in the public archive, so no extra install is needed.

Don't Transfer the Virtual Environment (.venv)

Run: Local (WSL)

Before transferring the application, exclude the local virtual environment directory `.venv`.

■ Why

- A virtual environment records absolute paths from when it was created (`pyvenv.cfg`, the activate script's `VIRTUAL_ENV`, executable shebangs, etc.). Copying it to another machine leaves it inconsistent, and it won't work correctly.
- On the server, you'll create a fresh virtual environment in Step 5. The local `.venv` isn't needed, and transferring it would also add transfer time since it contains many package files.

```
cd ~  
rm -rf rag-web-app/.venv
```

Note: Windows-origin Zone.Identifier files (alternate data streams) are harmless and can be ignored.

Step 4: Transfer the Application (scp)

Run: Local (WSL)

Syntax: `scp -r <source> <destination>` Which machine each side refers to depends on which side of the colon it's on.

```
cd ~  
scp -r ./rag-web-app ubuntu@<IP>:/tmp/
```

- **Where to run this:** Run this one level above rag-web-app (your home directory, ~). Specifying ./rag-web-app from inside that directory points to a path that doesn't exist and causes an error.
- **Destination /tmp:** A directory on the server. Writable by a regular user, so it's used for temporary handoff (the final location, /opt, requires root privileges).
- **Caution:** Using . as the source targets the entire current directory. Specify the transfer target by folder name instead.

Step 4 (cont.): Place It in the Right Directory

Run: Server (Ubuntu)

```
sudo mv /tmp/rag-web-app /opt/  
sudo chown -R ubuntu:ubuntu /opt/rag-web-app
```

- Move it from /tmp to /opt, and change its owner to the working user, ubuntu.
- [Keep the templates/ and static/ hierarchy intact \(Flask's default search location\).](#)

Directory Structure

```
/opt/rag-web-app/  
├─ app.py  
├─ requirements.txt  
├─ templates/  
│   └─ index.html  
└─ static/  
    ├── style.css  
    └─ app.js
```

Step 5: Create the Virtual Env and Install Dependencies

Run: Server (Ubuntu)

```
cd /opt/rag-web-app
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install gunicorn
```

- On the server, create a fresh virtual environment, then install requirements.txt (Flask, requests) and gunicorn.
- This step is why we didn't transfer .venv before Step 4.

Step 6: Set the Account Token

Run: Server (Ubuntu)

```
export AI_ENGINE_TOKEN="<your issued account token>"
```

- app.py reads the token from the AI_ENGINE_TOKEN environment variable.
- The value set by this command is only valid within that shell session.
- See the later slide for token-management principles.

Step 7: Start gunicorn and Verify It Works

Run: Server (Ubuntu)

```
gunicorn --workers 2 --bind 0.0.0.0:8000 app:app
```

- **app:app:** Refers to the application instance inside app.py.
- **0.0.0.0:8000:** Listens on port 8000 across all network interfaces, making it reachable from outside.

Check in your browser: <http://<server's global IP>:8000/> → If a question returns an answer and its sources, the deployment is complete.

Caution: The process stops when the SSH connection ends. Staying publicly available requires making the process persistent (see the next few pages).

Handling the Account Token

■ Principles

- Never include it in source code, HTML, JavaScript, or the repository (app.py only ever reads it from an environment variable).
- Supply it to the runtime environment as an environment variable.
 - Never log it.
- For version control, exclude .env and similar files via .gitignore.

Why: Because a leaked token leads directly to unauthorized use of the account.

Requirements for Production Use (Out of Scope Here)

Every item listed below is required when publishing a Web application for real-world production use. This material only covers verifying it works in a minimal setup, so these aren't covered in detail. Research them yourself and try implementing them.

Process Persistence / Auto-Start

Turn it into a service via systemd or similar. Auto-recovery after a restart.

Reverse Proxy

Publish on ports 80/443 via Apache / nginx.

Packet Filter

Controlling inbound traffic on SAKURA Cloud.

Host Firewall

OS-level traffic control via ufw or similar.

Encrypting Traffic

HTTPS / TLS (Let's Encrypt, etc.).

Restricting Admin Access

Limit SSH to public-key auth, restrict source IPs.

Maintenance

Regular updates to the OS and dependencies.

Monitoring / Logging

Uptime monitoring and log management.

Summary

We set up a Flask-based RAG Web application to return HTTP responses on a SAKURA Cloud Ubuntu server.

■ Key Points for Running It on a Server

- Start it with the production WSGI server gunicorn, listening on 0.0.0.0 to expose it externally.
 - (the dev server you used locally, python app.py, isn't used).
- Code alone isn't enough — on the server, you also need a virtual environment and dependencies in place, with the token supplied as an environment variable.

■ Things to Watch For During Implementation

- Confirm which machine you're on (local / server) from the prompt.
- Don't transfer the virtual environment (.venv); create it fresh on the server.
- Run scp one level above the target folder, naming the folder explicitly.